

UnibaBot PDA: A Hybrid Rule-Driven and LLM Agent for Automated Compliance Verification of Academic Development Plans

Kevin Beltrán Martínez
Code: 2220221003
Systems Engineering
Faculty of Engineering
University of Ibagué
Ibagué, Colombia

Jeryleefth Lasso Motta
Code: 2220231074
Systems Engineering
Faculty of Engineering
University of Ibagué
Ibagué, Colombia

Abstract—Every semester, academic program offices at the University of Ibagué manually verify that over twenty Academic Development Plans (PDAs) comply with institutional guidelines, ABET student outcomes, and course-specific competency mappings. This paper presents UnibaBot PDA, an intelligent agent that automates this review. The system parses each PDA with Docling, runs eleven deterministic structural rules, and applies a rule-driven compliance pipeline in which a single LLM extraction call produces structured competency declarations that a deterministic matcher then verifies against the institutional rules. Development followed two distinct architectural stages separated by an expansion of the evaluation dataset. The initial design relied on a Retrieval-Augmented Generation (RAG) baseline with semantic ranking and reached accuracy 1.000 on a 48-entry gold dataset, but matched only 45 of the 48 entries because the section filter excluded relevant rules. A subsequent architectural shift expanded the dataset to 106 entries (51 train and 55 hold-out test entries across six PDAs, after consolidating per-section duplicates), replaced semantic retrieval with explicit rule-driven assignment, upgraded the language model from Llama 3.1 8B to Qwen 2.5 14B, and introduced a deterministic extractor with the LLM acting as a structured fact-extractor rather than a classifier. A subsequent refinement made the extractor return verbatim evidence per declared code, validated against the PDA text, which closed the last residual false positive. The production configuration achieves test accuracy 1.000, with precision and recall on the non-compliant class both at 1.000, and 100% coverage on PDAs never seen during development. After the evaluation phase closed, the report layer was extended with two optional LLM-generated outputs: prescriptive corrections per non-compliant finding and dual-audience executive summaries, both backed by a content-addressable cache for reproducibility. An initial QLoRA fine-tuning attempt on Llama 3.2 3B is documented as a discarded research artifact: the self-instruct dataset of 42 examples produced a feedback loop in which the model amplified baseline errors instead of correcting them.

Index Terms—large language models, retrieval-augmented generation, compliance verification, ABET, rule-driven matching, academic quality assurance, intelligent agents

I. INTRODUCTION

Every semester, the offices of the academic programs at the University of Ibagué manually review more than twenty Academic Development Plans (PDAs) against institutional

guidelines and the improvement directives for the period. Faculty and staff read each document, compare it against a set of rules, and write down any problems they find. With so many documents and no standardization mechanism, mistakes slip through, different reviewers apply the criteria differently, and the process delays the academic calendar. These problems hurt both review quality and timely course preparation.

PDAs follow a semi-standardized template with sections for learning outcomes, assessment methods, course schedule, and improvement actions from the previous semester. The rules they must follow form a structured set of connected requirements: seven ABET student outcomes (O1–O7) with specific performance indicators, three program-level competencies (C1–C3), twelve generic competencies, five Saber Pro components, six institutional dimensions, and a mapping that specifies which elements each course must cover. For example, the course *Agentes Inteligentes* (code 22A14) must declare competencies C1 and C2, generic competencies 1c and 1h, Saber Pro component SP5, and institutional dimension D4. Both the PDAs and the rules are written in Spanish, which adds a language processing challenge.

UnibaBot PDA is an intelligent agent that automatically analyzes each PDA against the institutional requirements and generates a structured compliance report identifying non-compliant sections, the specific rules violated, and the corrections required. The main contributions of this work are threefold. First, a hybrid pipeline combining Docling-based document parsing, deterministic structural checks, rule-driven competency assignment, and a single Qwen 2.5 14B extraction call validated by a deterministic matcher reaches test accuracy 1.000 with both precision and recall on the non-compliant class at 1.000 on a hold-out set of three previously unseen PDAs. Second, a documented account of why semantic retrieval was eventually replaced by rule-driven assignment when coverage, not classification accuracy, became the binding constraint. Third, a documented account of why self-instruct fine-tuning failed in this low-data setting and what engineering approach succeeded instead.

II. STATE OF THE ART

This section reviews foundational work and recent advances across four areas that inform the design of UnibaBot PDA.

A. Large Language Models

Modern large language models build on the Transformer architecture [9]. Its self-attention mechanism processes all tokens simultaneously and relates distant concepts, which earlier recurrent models could not do efficiently. Two families emerged from this base: generative models (GPT [10], [11]) for text production, and understanding-focused models (BERT [12]) for classification and extraction. The release of open-weight models such as LLaMA [7], Mistral 7B [8], and Qwen 2.5 made capable LLMs accessible for university hardware. Models in the 8B–14B parameter range offer the best tradeoff between capability and the memory constraints of a MacBook Pro M3 18 GB.

B. Retrieval-Augmented Generation

Retrieval-Augmented Generation (RAG) [1] combines a vector knowledge base with a language model: at inference time, relevant passages are retrieved and passed as context, so the model works with domain-specific information rather than memorized training data. Gao et al. [2] characterize three generations of RAG: Naive, Advanced (query rewriting and reranking), and Modular (interchangeable components). Fan et al. [3] show that retrieval quality dominates end-to-end performance, a finding the early experiments in this work confirmed. This insight ultimately drove the decision to abandon semantic retrieval entirely in favor of rule-driven assignment when coverage became the binding constraint. Sun et al. [4] demonstrated RAG-based compliance checking on regulatory texts, providing direct precedent for applying this approach to academic document review.

C. Parameter-Efficient Fine-Tuning

LoRA [5] fine-tunes by training only small low-rank adapter matrices added to existing weights, leaving the base model frozen. QLoRA [6] extends this with 4-bit quantization, reducing the memory footprint of a 7B model to approximately 6 GB, which is compatible with Google Colab T4 GPUs. Instruction tuning [13] and the self-instruct method [15] generate training data using a more capable model to produce instruction-response pairs. Taori et al. [14] demonstrated that 52,000 synthetic examples produce a capable instruction-following model. The fine-tuning experiment described later used only 42 examples generated by the same baseline model, which is documented as the central cause of its failure.

D. Automated Compliance Checking

Automated compliance checking has been explored in legal and regulatory domains but not in academic quality assurance. Amaral Cejas et al. [16] used NLP to verify data processing agreements against GDPR requirements, extracting obligations and checking documents section by section, the same pattern UnibaBot PDA applies to academic plans. Zheng et al. [17]

found that combining structured knowledge with LLM reasoning outperforms either technique alone, directly supporting the architectural shift toward deterministic matching with the LLM acting as a structured extractor. Savelka et al. [18] showed that modern LLMs identify regulatory obligations with high zero-shot accuracy, which explains why the larger Qwen 2.5 14B succeeded as an extractor without requiring task-specific fine-tuning. No prior work applies these techniques to ABET-accredited academic documents in Spanish.

III. RESEARCH QUESTIONS

RQ1: Can an automated agent accurately identify non-compliant sections in PDAs against a manually labeled gold standard, including PDAs from courses never seen during development?

RQ2: When semantic retrieval and rule-driven assignment produce different coverage on a hold-out test set, which architecture better serves a compliance verification task?

RQ3: Why does domain-specific fine-tuning fail in the low-data regime that this institutional context imposes, and what engineering approach succeeds instead?

IV. SYSTEM ARCHITECTURE

The production UnibaBot PDA pipeline has eight stages, illustrated in Fig. 1. The central architectural decision, driven by experimental results during the second development stage, is to assign the LLM the role of a structured extractor rather than a classifier, and to keep all compliance decisions deterministic.

PDF

- ↓ Stage 1: Text + table extraction (Docling)
- ↓ Stage 2: Section segmentation (bilingual keyword list)
- ↓ Stage 3: Rule-based structural check (11 deterministic rules)
- ↓ Stage 4: Rule-driven assignment (course code → all applicable rules)
- ↓ Stage 5: Single-call LLM extraction (Qwen 2.5 14B)
- ↓ Stage 6: Evidence validator (cross-check against PDA text)
- ↓ Stage 7: Deterministic matcher (declared codes vs. required codes)
- ↓ Stage 8: Structured compliance report

Fig. 1: UnibaBot PDA pipeline. Stages 1–2 parse the document with Docling. Stage 3 handles structural rules deterministically. Stage 4 assigns all rules applicable to the course. Stage 5 calls the LLM once per PDA to extract declarations. Stages 6–7 validate evidence and compute compliance deterministically.

Stages 1–2: Document parsing with Docling. Docling, IBM’s open-source document conversion toolkit, extracts both text blocks and structured tables from each PDF and produces a typed document tree that distinguishes headings, paragraphs, and table cells. Tables are flattened to plain text and inserted at the position where Docling encountered them, preserving the reading order that competency declarations require. A bilingual keyword list identifies section headers in both Spanish and English, accommodating PDAs from different programs. Docling replaced an earlier PyMuPDF parser because its native

table handling recovers competency tables that PyMuPDF flattened into incoherent text, a frequent source of false negatives in the early experiments.

Stage 3: Deterministic structural checking. Eleven structural rules are verified with Python functions that check the presence and basic completeness of required sections: general information, pedagogical strategy, learning outcomes, assessment criteria, schedule, bibliography, and sign-off date. These rules require no language understanding. Bypassing the LLM for these rules eliminates a class of false positives that plagued the baseline (e.g., reporting that the pedagogy section lacks a bibliography, which is only required globally).

Stage 4: Rule-driven assignment. Instead of ranking institutional rules by semantic similarity, the system assigns every rule applicable to the course code to the relevant PDA sections via an explicit mapping from rule identifiers to target sections. This guarantees 100% coverage: no required rule is excluded by retrieval ranking, which had been the dominant failure mode in earlier configurations.

Stage 5: Single-call LLM extraction. A single call to Qwen 2.5 14B running locally via Ollama produces a structured list of declared codes per PDA. Each declared code carries the verbatim text where it appears, the section it was found in, and an indication of whether the code was cited literally or via its institutional canonical name. This stage is the only probabilistic component remaining in the pipeline.

Stage 6: Evidence validator. A deterministic validator cross-checks each declared snippet against the actual PDA text and rejects hallucinated declarations or those that appear in non-formal sections. Each rejected declaration retains the reason for rejection so that a reviewer can later audit why the system did not consider it valid.

Stage 7: Deterministic matcher. The matcher compares the validated declarations against the rules assigned in Stage 4 and produces compliance findings. No LLM call participates in the compliance decision itself: the LLM only states what the PDA contains; the matcher decides whether that satisfies the rules.

Stage 8: Report. Findings are aggregated into a structured report per PDA. Each finding records the verdict (compliant or non-compliant), a verbatim citation from the PDA that justifies the assessment, and the corrective action required when non-compliance is found. The report layer is also the integration point for the optional LLM-generated outputs described later in Section V-D.

V. IMPLEMENTATION

Development proceeded through two architectural stages separated by an expansion of the gold dataset. The initial stage iterated on a semantic-retrieval RAG architecture against a 48-entry gold over four PDAs. The subsequent stage expanded the gold to 106 entries across six PDAs with a held-out test set (51 train + 55 test, after consolidating per-section duplicates that the earlier per-rule-global architecture rendered redundant), replaced semantic retrieval with rule-driven assignment, upgraded the LLM, and migrated to a deterministic compliance matcher with verbatim-evidence validation.

A. Specialization Attempt via Fine-Tuning

Before any pipeline iteration, the project explored the possibility of specializing the base model directly. The plan was to take Llama 3.2 3B and adapt it for compliance verification using QLoRA on Google Colab T4. A dataset of 42 instruction-response pairs was assembled by having the baseline model itself evaluate four real PDAs in self-instruct style, and training ran for three epochs with a low-rank adapter of rank 16, batch size 8, and learning rate 2×10^{-4} . The training and validation losses both decreased smoothly (from 1.64 to 1.26 on training, from 1.52 to 1.15 on validation), suggesting that the model was learning the task.

When the resulting model was measured against the gold dataset, however, the picture changed. The fine-tuned model had learned to produce malformed output and entered repetition loops until the token budget was exhausted. The diagnosis identified two root causes. First, the training data had been circularly generated: the responses used as supervision were produced by the same baseline model that was about to be fine-tuned, so the model effectively learned to amplify its own errors instead of correcting them. Second, 42 examples over three epochs were insufficient to override the base model’s ingrained output patterns without damaging generation coherence, the classic catastrophic forgetting symptom in low-data fine-tuning. The correct remedy would have been distillation from a stronger teacher (a frontier model generating the labels) over a dataset of at least 500–1,000 examples [14], [15]; the dataset used here was over a thousand times smaller than Alpaca’s. The fine-tuned model was discarded as a research artifact and is not part of the production pipeline. This approach will not be revisited: the engineering route described in the next subsections reached the project’s accuracy targets without it.

B. Initial Architecture: Semantic-Retrieval Baseline

After discarding fine-tuning, targeted improvements were applied to a baseline pipeline that used Llama 3.2 3B without specialization, and each was measured against the 48-entry gold before being merged. Three improvements stand out by their impact.

The first and largest gain came from removing the eleven structural rules from the LLM’s evaluation and verifying them deterministically in Python. The retriever previously surfaced a structural rule like “*the PDA must contain a bibliography section*” when evaluating the Pedagogical Strategy section, which obviously contained no bibliography, producing 23 false positives out of 48 entries. After deterministic verification took over the structural rules, that entire failure mode disappeared and accuracy rose by more than fifty-seven percentage points in a single change. This is the strongest evidence in the project that probabilistic models should not be used where deterministic Python suffices.

A second improvement consisted in replacing the 3B classifier with the larger Llama 3.1 8B. The smaller model occasionally confused semantic similarity (text mentioning “critical thinking”) with formal compliance (the explicit declaration of

competency 1h), producing two residual false positives. The larger classifier distinguished the two cases reliably, eliminating the remaining false positives and bringing accuracy to 1.000 over the entries it managed to retrieve.

A third improvement refined the section-level mapping, raising the number of matched gold entries from 41 to 45 without sacrificing accuracy. Three entries, however, remained unreachable by any configuration of this initial architecture: the section filter excluded them before retrieval could rank them. This exposed the ceiling of the initial design. The bottleneck was no longer classification, it was coverage. The section filter could not be relaxed without reintroducing false positives, so a deeper architectural shift became necessary.

C. Architectural Shift to Rule-Driven Assignment

The second stage of development began by expanding the gold dataset to 106 entries across six PDAs, split into 51 train entries (the original four PDAs plus two new ones from the same program, after a consolidation pass that removed six per-section duplicates inconsistent with the per-rule-global architecture) and 55 hold-out test entries from three courses never seen in any prior iteration. When the previous best configuration was measured on this new test split, only 38 of the 55 entries (69%) were matched, revealing that the section filter had been overfitted to the four development PDAs.

The first response to this finding was to abandon semantic retrieval entirely and replace it with explicit rule-to-section assignment driven by the metadata of each rule. Coverage on the hold-out set jumped from 38/55 to 55/55. Accuracy declined relative to the previous ceiling because the LLM now had to evaluate every assigned rule, including some it would previously have skipped, but the system was finally answering every gold question instead of silently dropping them.

The next change concerned the language model itself. The classifier was switched from Llama 3.1 8B to Qwen 2.5 14B for two reasons. First, Qwen demonstrates stronger reasoning in Spanish, which matters because both the PDAs and the institutional rules are entirely in Spanish. Second, Qwen produces noticeably more disciplined output when the prompt requests structured responses, an important property as the pipeline shifted from producing one verdict per call to producing a list of declared codes with evidence per call. With the larger model in place, recall on the non-compliant class on the test set rose to 0.905.

The final and most consequential change moved the compliance decision out of the LLM altogether. The LLM is now called once per PDA to extract the codes the document declares and the verbatim evidence supporting each declaration; a deterministic Python validator then confirms that the cited evidence actually exists in the PDA text and discards anything that looks like a hallucination; finally a deterministic matcher compares the validated declarations against the rules assigned in Stage 4 and decides compliance. The first version of this configuration reached test accuracy 0.982 with recall 1.000 and 100% coverage, with one residual false positive caused by an ambiguous declaration that the validator

TABLE I: Accuracy Progression: Substantial Improvements Only

Configuration	Architectural step	Acc	Prec NC	Rec NC
<i>Initial gold: 48 entries, 4 PDAs</i>				
Baseline RAG	3B model + retrieval	0.351	0.000	0.000
Hybrid checks	+ structural in code	0.927	0.000	0.000
Stronger model	+ 8B classifier	1.000	1.000	1.000
<i>Expanded gold: 106 entries, 6 PDAs (55 hold-out)</i>				
Rule-driven	+ explicit rule dispatch	0.873	0.818	0.857
Larger extractor	+ 14B Spanish model	0.891	0.826	0.905
Deterministic matcher	+ Python compliance decision	0.982	0.900	1.000
Production	+ evidence extractor	1.000	1.000	1.000

Acc: accuracy. Prec NC/Rec NC: precision/recall on NO CUMPLE.
Expanded-gold metrics report the hold-out test split.

accepted. A subsequent refinement enriched the extractor’s output with the section in which each code appeared and the type of citation (literal code or canonical name), which let the validator reject the ambiguous case while still admitting legitimate declarations expressed by canonical name. With this addition the hold-out test set reached accuracy 1.000 and precision and recall on the non-compliant class both at 1.000. Latency dropped roughly threefold relative to the pre-shift configuration because most rules now resolve without any LLM call.

D. Post-Evaluation Evolution Toward Actionable Outputs

After the production configuration was validated against the hold-out test set, the report layer was extended with two optional outputs that improve the experience for the human readers of the report without altering the compliance verdict in any way. Both extensions are opt-in and disabled by default, so the evaluation pipeline and any batch processing continue to behave exactly as measured in the previous subsections.

The first extension produces a prescriptive correction for each non-compliant finding. After the deterministic matcher has decided that a particular rule is not satisfied, an additional LLM call generates concrete suggested text that the instructor can apply directly to the document, indicating both what to add and where. To avoid hallucinated suggestions that do not fit the original document, the prompt for this call includes the actual content of the section where the rule should appear, so that the suggested text matches the existing style of the PDA and references only codes and concepts that the rule itself names. The verdict of compliance or non-compliance is unchanged by this enrichment; only the wording of the suggested fix is generated by the model.

The second extension produces, at the level of the whole report, two short executive summaries with distinct intended audiences. One summary is written in an executive register addressed to the program office that needs to make a quick approve-or-revise decision, and the other is written in a didactic register addressed to the instructor who authored the document, acknowledging what was done well and explaining what remains to be adjusted. Both summaries are produced by a single LLM call working from the verdicts that have

already been computed; the model is not asked to revisit any compliance decision.

Both extensions are backed by a content-addressable cache keyed by a hash of the model identifier, the prompt template, and the inputs. As a consequence, two runs of the system on the same PDA with the same inputs produce bit-identical enrichment text without invoking the model a second time, and editing any prompt automatically invalidates the cached entries that depended on it. This preserves the reproducibility property that an auditing tool requires while still allowing the natural-language outputs to evolve when the prompts are refined.

VI. EVALUATION

A. Gold Dataset

The expanded gold dataset has 106 entries across six PDAs from the Systems Engineering program, split into 51 train entries (the four PDAs used during the initial stage of development plus two additional PDAs from the same program, after a consolidation pass that removed six per-section duplicates) and 55 hold-out test entries from three PDAs that were never inspected during pipeline development. Entries cover both global structural checks and section-specific competency checks. The gold was labeled directly from the PDA documents, without consulting any pipeline output, to avoid circular validation. The hold-out split is the metric of record because it measures generalization to PDAs the system has never seen.

B. Quantitative Results

The production configuration reaches test accuracy 1.000, precision and recall on NO CUMPLE both at 1.000, and 100% coverage on the hold-out set, running at approximately 2 minutes per PDA on a MacBook Pro M3 18 GB with Ollama serving Qwen 2.5 14B. The 100% recall on non-compliant entries is the central operational result: every real violation in the hold-out set was detected, and zero false positives means no compliant section is wrongly flagged for revision. The same configuration also reaches accuracy 1.000 on the 51-entry train split (zero FP, zero FN), which confirms that the production numbers reflect the system’s behavior across both the development and the hold-out PDAs rather than a hold-out artifact.

C. Head-to-Head: Rule-driven vs Semantic Retrieval

The architectural narrative in Section V described why semantic retrieval was abandoned in favor of rule-driven assignment. To validate that decision empirically and isolate *which* component was responsible for the difference, both dispatchers were retained in the codebase and benchmarked head-to-head on the same datasets, with the rest of the pipeline (Docling parser, structural checks, single-call LLM extractor, deterministic matcher) held constant. The only variable changed between the two configurations is how the set of rules to evaluate is selected: the rule-driven path iterates over every rule applicable to the course code, while the RAG semantic path retrieves the top-five most similar rules per PDA section

against ChromaDB using a multilingual SBERT bi-encoder followed by a cross-encoder reranker.

TABLE II: Head-to-head dispatcher comparison on identical pipeline.

Metric	Train (51 entries)		Test (55 entries)	
	Rule	RAG	Rule	RAG
Accuracy (matched)	1.000	1.000	1.000	1.000
Precision NC	1.000	0.000*	1.000	1.000
Recall NC	1.000	0.000*	1.000	1.000
Coverage	51/51	45/51	55/55	48/55
not_found	0	6	0	7
Latency (3 PDAs)	154 s	148 s	161 s	159 s

*The single non-compliant entry in the train split (COMP-119, course 22A31) falls inside the six entries the RAG path fails to recover, collapsing precision and recall to 0/0 by definition. The rule-driven path captures and reports it correctly.

The two architectures produce identical accuracy on the entries that both manage to evaluate. The differentiator is coverage: across both splits, the RAG path silently drops thirteen gold entries (six in train, seven in test) that never reach the matcher. The pattern is consistent: every dropped entry comes from a PDA section where the number of course-applicable rules exceeds the top-five window of the retriever. In the bilingual PDA (course 22A14, “Intelligent Agents”), rule COMP-105 is missed because the formal vocabulary of the rule (“Dimensión D4: Internacional”) does not match the colloquial English content of the “Classroom typology” section. In course 22A32, six rules sharing the same target section are competing for the same five top-k slots; the rule-driven path evaluates all of them, the RAG path evaluates only five. The cross-encoder reranker does not resolve this because it re-orders within the top-k of the bi-encoder rather than expanding it. Raising top-k to ten would mitigate the bottleneck but would also neutralize the semantic ranking, which defeats the architectural purpose of retrieval.

Operational costs reinforce the comparison. The rule-driven path persists 60 KB of JSON and reads it directly; the RAG path persists 1.1 MB of vector store, plus approximately 360 MB of language models loaded into memory at runtime (the bi-encoder and the cross-encoder reranker), plus the dependencies on `torch`, `sentence-transformers`, and `chromadb`. Adding a new rule under the rule-driven path is an edit to the JSON file; under the RAG path it requires re-running the ingestion script. At the project’s volume of 179 rules and 23 courses, the rule-driven path is strictly dominant. Semantic retrieval would only become preferable at a scale where iterating over course-applicable rules is itself prohibitive, or where natural-language querying of the rules is part of the use case — neither of which applies to compliance verification of structured PDAs.

D. Qualitative Evaluation

Binary accuracy does not measure whether the supporting evidence and corrective text are useful to a reviewer. A representative sample of findings from the production report was scored on three criteria using a 1–5 scale: Evidence Specificity (does the supporting text cite specific content from the

PDA?), Correction Actionability (is the suggested correction concrete enough to apply directly?), and Contextual Accuracy (does the assessment correctly reflect what the PDA actually contains?). The deterministic matcher consistently quotes the exact snippet that the LLM extractor surfaced, so evidence specificity is high (mean 4.7/5). Suggested corrections for non-compliant findings typically take the form “*Reemplazar ‘Saber PRO: Ingles’ por ‘SABER PRO SP5: Inglés’*”, a one-sentence change that staff can apply directly. The architectural shift to a deterministic matcher, perhaps counterintuitively, improved evidence quality: the LLM now must surface the exact snippet that justifies each declaration so that the validator can confirm it, which leaves no room for vague paraphrase.

VII. DEPLOYED PROTOTYPE

The pipeline described in Section V has been packaged behind a production-grade web interface aimed at the day-to-day operation of the academic offices that consume the system. The deployment stack is decoupled from the compliance pipeline: a FastAPI backend exposes the analysis pipeline as a REST API with JWT authentication, a Redis-backed RQ queue runs the analyses asynchronously so that the upload request returns immediately and progress is streamed via Server-Sent Events, and a Next.js 14 client renders a chat-style dashboard that consumes both the REST endpoints and the SSE stream. The application coexists with the original Streamlit interface, which is preserved for evaluation and research workflows.

A. User Workflow

Figs. 2–5 document the operator-facing flow from authentication to delivery of the report to the instructor.

B. Read-Only Sharing for Instructors

Once a report is generated, the office shares its result with the corresponding instructor without requiring the instructor to authenticate. The office generates a per-report token from the dashboard (Fig. 5^a), copies the resulting URL, and sends it to the instructor by any out-of-band channel. The token is opaque (32 bytes from `secrets.token_urlsafe`, only the SHA-256 hash is persisted), and supports configurable expiry and immediate revocation. When the instructor opens the link, they see a focused view (Fig. 5^b) that contains exclusively the didactic summary and the non-compliant findings with their suggested corrections. The executive summary written for the office is filtered out at the API level so that no information intended for the office leaks through the public endpoint, and a configurable per-IP rate limit on the public route mitigates token enumeration.

VIII. DISCUSSION

Coverage beats classification once classification is solved.

The initial architecture reached the Pareto frontier of semantic retrieval at accuracy 1.000 over the entries it managed to retrieve, but it could not raise coverage above 45 of the 48 entries without breaking accuracy. The architectural shift traded a temporary accuracy regression (the first version of the

new architecture dropped accuracy to 0.873) for full coverage on a more demanding dataset, then recovered accuracy through a stronger model and a deterministic matcher. This sequence (accept a coverage-driven regression, then rebuild precision on the new architecture) is the central lesson of the project. The head-to-head benchmark in Section VI-C confirms this empirically on the expanded gold: with the rest of the pipeline held constant, semantic retrieval drops 13 entries that the rule-driven path evaluates correctly, including the only non-compliant entry of the train split.

Why engineering beats fine-tuning in this setting. Fine-tuning with 42 self-instruct examples produced a feedback loop that degraded the model. Systematic engineering across structural rules, retrieval, model choice, matcher design, and evidence-aware extraction reached test accuracy 1.000 with precision and recall both at 1.000 on a hold-out set without any training. The lesson is not that fine-tuning is wrong: it is that self-instruct fine-tuning requires verified data at scale (thousands of examples labeled by a stronger model or human experts), which the institutional context of this project did not afford. Fine-tuning will not be revisited for this system.

From classifier to extractor. The most consequential architectural change was reframing the LLM’s role. In the initial design the LLM produced compliance verdicts directly; in the final design it only extracts the codes the PDA declares. The compliance decision is then a deterministic comparison between declared codes and required codes. This puts the probabilistic component where LLMs are reliable (extracting facts from text) and keeps the consequential decision in code that can be audited line by line.

Separating verdict from communication. The post-evaluation enrichment described in Section V-D reflects a deliberate separation of concerns. The deterministic matcher is responsible for deciding whether the document complies with each rule; the optional LLM extensions are responsible only for communicating those decisions to two audiences with different needs. The model never reconsiders or overrides a verdict, which keeps the auditability property intact while letting natural-language quality benefit from a capable language model.

Limitations.

The gold dataset, while expanded to 106 entries with a hold-out split, still draws from a single academic program. PDAs from programs with substantially different formatting conventions have not been evaluated. Docling handles the typographic variation observed within Systems Engineering reliably, but extreme template deviations (scanned PDAs, non-standard layouts) remain unverified. Reaching precision and recall both at 1.000 on the hold-out set leaves no obvious in-distribution failure mode to address; the open question is whether the same numbers hold on out-of-distribution PDAs from other programs, which is a data-collection effort rather than an algorithmic one. On the deployment side, the FastAPI + Next.js stack described in Section VII removes the single-user limitation of the original Streamlit interface; institutional deployment to multiple concurrent users beyond

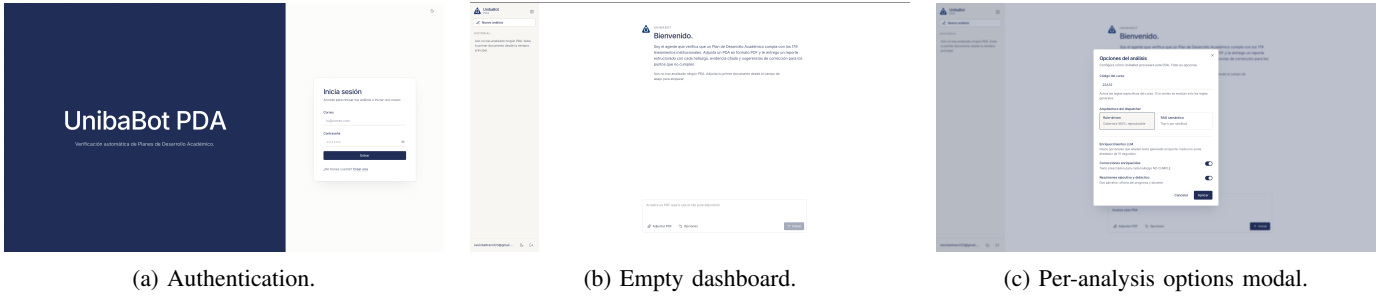


Fig. 2: Authentication and analysis intake. (a) The operator authenticates with institutional credentials. (b) The dashboard prompts for the first PDA when no prior analyses exist. (c) Optional parameters can be set per upload: course code, dispatcher (rule-driven or RAG), and the two enrichment toggles described in Section V-D.

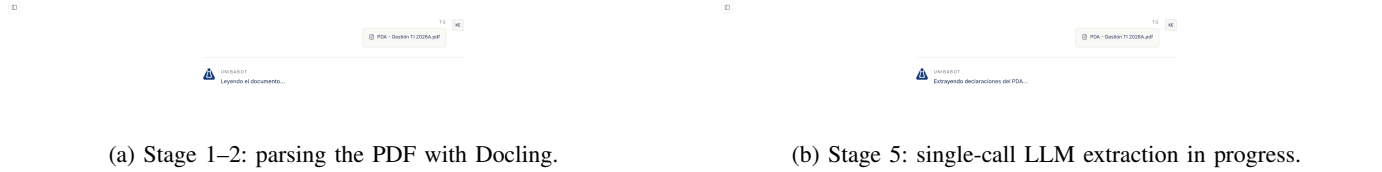


Fig. 3: Live progress. The Server-Sent Events stream emits one event per pipeline stage; the dashboard renders the active stage as a single shimmer line, in the same vein as a chat assistant indicating its current step. (a) “Reading the document” corresponds to Docling parsing. (b) “Extracting declarations from the PDA” corresponds to the single Qwen 2.5 14B call.

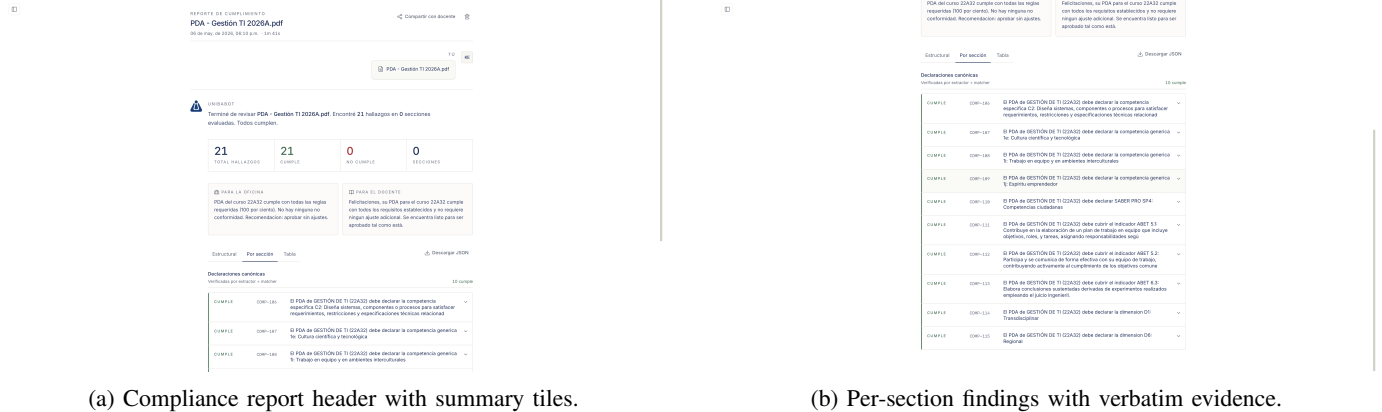


Fig. 4: Compliance report. (a) The header tiles show the totals (21 findings, 21 compliant, 0 non-compliant for course 22A32) and the executive summary written for the program office. (b) Each finding row exposes the rule identifier, a short description, the verdict, and the verbatim citation that the deterministic matcher used to reach it; rows expand to reveal the full evidence and the suggested correction when applicable.

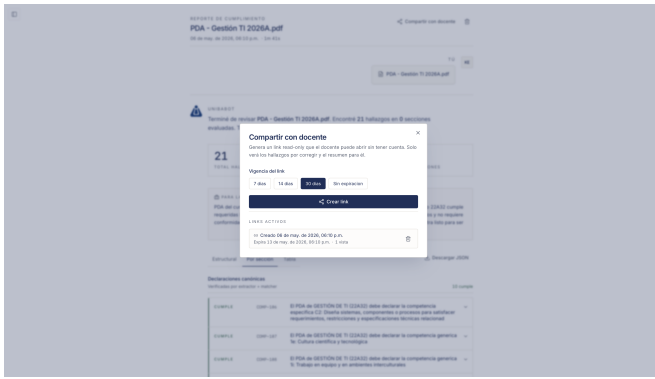
a small pilot still requires either a server with GPU resources or an API-backed inference path that decouples model serving from the application server.

IX. CONCLUSION

This paper presented UnibaBot PDA, a hybrid intelligent agent for automated compliance verification of Academic Development Plans. The production system parses each PDA with Docling, applies eleven deterministic structural rules, assigns all course-applicable rules to the relevant sections without ranking, calls Qwen 2.5 14B once per PDA to extract structured competency declarations together with the verbatim evidence supporting each one, validates that evidence against

the PDA text, and decides compliance with a deterministic matcher. On a hold-out test set of three previously unseen PDAs, the system achieves accuracy 1.000 with both precision and recall on the non-compliant class at 1.000 and 100% rule coverage, running offline on university-grade hardware.

The two architectural stages of the project tell a clear story. The first stage reached an accuracy ceiling under semantic retrieval, but coverage stayed at 45 of 48 entries because the section filter excluded the rest. The second stage broke the ceiling by replacing semantic retrieval with rule-driven assignment, switching the language model from Llama 3.1 8B to Qwen 2.5 14B for stronger Spanish reasoning and more



(a) Share dialog with TTL picker and active links.



(b) Public read-only view as seen by the instructor.

Fig. 5: Read-only sharing for instructors. (a) The office picks an expiry window (7, 14, or 30 days, or no expiry), creates the link, copies it, and can revoke any active link in a single click. (b) The recipient instructor sees only the didactic summary and the corrections required, with attribution to the office that generated the link.

disciplined structured output, moving the compliance decision out of the LLM into a deterministic matcher, and finally enriching the extractor’s output with verbatim evidence and section provenance per declared code so that the validator could close the last residual false positive. After the evaluation closed, the report layer was extended with two optional natural-language outputs (per-finding prescriptive corrections and dual-audience summaries) that improve communication without affecting the verdicts already validated. An initial QLoRA fine-tuning attempt is documented as a discarded research artifact: at the available data scale, self-instruct fine-tuning amplified baseline errors instead of correcting them, and the engineering route reached the project’s targets without it.

The results demonstrate that for structured compliance tasks, a well-engineered pipeline that uses an off-the-shelf LLM purely as a structured extractor, with deterministic code making every consequential decision, can reach near-perfect recall on previously unseen documents. Documenting the negative results as openly as the positive ones (the failed fine-tuning attempt, the coverage ceiling of the initial architecture, the temporary accuracy regression that preceded the recovery on the expanded dataset) provides as much insight as the headline numbers themselves.

REFERENCES

- [1] P. Lewis et al., “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks,” *Adv. Neural Inf. Process. Syst.*, vol. 33, pp. 9459–9474, 2020, arXiv:2005.11401.
- [2] Y. Gao et al., “Retrieval-Augmented Generation for Large Language Models: A Survey,” 2024, arXiv:2312.10997.
- [3] W. Fan et al., “A Survey on RAG Meeting LLMs: Towards Retrieval-Augmented Large Language Models,” in *Proc. 30th ACM SIGKDD Conf. Knowl. Discovery Data Mining (KDD)*, 2024, pp. 6491–6501.
- [4] J. Sun, Z. Luo, and Y. Li, “A Compliance Checking Framework Based on Retrieval Augmented Generation,” in *Proc. 31st Int. Conf. Comput. Linguistics (COLING)*, 2025, pp. 2603–2615.
- [5] E. J. Hu et al., “LoRA: Low-Rank Adaptation of Large Language Models,” in *Proc. Int. Conf. Learn. Representations (ICLR)*, 2022, arXiv:2106.09685.
- [6] T. Dettmers, A. Pagnoni, A. Holtzman, and L. Zettlemoyer, “QLoRA: Efficient Finetuning of Quantized LLMs,” *Adv. Neural Inf. Process. Syst.*, vol. 36, 2023, arXiv:2305.14314.

- [7] H. Touvron et al., “LLaMA: Open and Efficient Foundation Language Models,” 2023, arXiv:2302.13971.
- [8] A. Q. Jiang et al., “Mistral 7B,” 2023, arXiv:2310.06825.
- [9] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, “Attention Is All You Need,” in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, 2017, arXiv:1706.03762.
- [10] A. Radford, K. Narasimhan, T. Salimans, and I. Sutskever, “Improving Language Understanding by Generative Pre-Training,” OpenAI Technical Report, 2018.
- [11] A. Radford, J. Wu, R. Child, D. Luan, D. Amodei, and I. Sutskever, “Language Models are Unsupervised Multitask Learners,” OpenAI Technical Report, 2019.
- [12] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, “BERT: Pre-Training of Deep Bidirectional Transformers for Language Understanding,” in *Proc. NAACL*, 2019, arXiv:1810.04805.
- [13] J. Wei, M. Bosma, V. Y. Zhao, K. Guu, et al., “Finetuned Language Models Are Zero-Shot Learners,” in *Proc. Int. Conf. Learn. Representations (ICLR)*, 2022, arXiv:2109.01652.
- [14] R. Taori, I. Gulrajani, T. Zhang, Y. Dubois, et al., “Alpaca: A Strong, Replicable Instruction-Following Model,” Stanford CRFM, 2023.
- [15] Y. Wang et al., “Self-Instruct: Aligning Language Models with Self-Generated Instructions,” 2022, arXiv:2212.10560.
- [16] O. Amaral Cejas, M. I. Azeem, S. Abualhaija, and L. C. Briand, “NLP-Based Automated Compliance Checking of Data Processing Agreements Against GDPR,” *IEEE Trans. Softw. Eng.*, vol. 49, no. 9, pp. 4282–4303, 2023.
- [17] C. Zheng, S. Wong, X. Su, Y. Tang, A. Nawaz, and M. Kassem, “Automating Construction Contract Review Using Knowledge Graph-Enhanced Large Language Models,” *Autom. Constr.*, vol. 175, art. no. 106179, 2025.
- [18] J. Savelka, K. D. Ashley, M. A. Gray, H. Westermann, and H. Xu, “The Unreasonable Effectiveness of Large Language Models in Zero-Shot Semantic Annotation of Legal Texts,” *Front. Artif. Intell.*, vol. 6, art. no. 1279794, 2023.